S&DS 265 / 565

Introductory Machine Learning

Neural Networks

March 23

Yale

Version of 2026/03/22 at 20:16

Welcome back!

For today:

- Where have we been? Where are we going?
- Neural networks

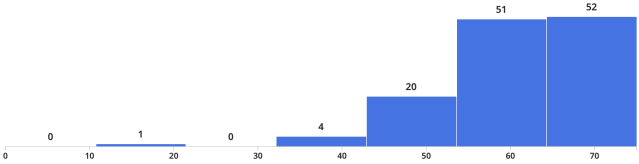
But first...

- Midterms posted, mid-semester grades announced
- Assignment 3 due this Thurs, March 26
- Assignment 4 out at same time

Midterm distribution

Review Grades for Midterm exam

● Regrade Requests Open ● Grades Not Published



Minimum	Median	Maximum	Mean	Std Dev
19.5	62.25	73.5	60.55	9.15

170 Students

Invitation: Panel discussion

Class on April, 20

We will have a panel discussion on

Societal issues for AI and Machine Learning

If you would like to volunteer to participate, please email us:

```
To: sds265@yale.edu
Subject: iML panel
```

Participants will receive a thank you gift! :-)

Where we've been

1	Jan 12, 14	Course overview; Python and background concepts	Python elements Covid trends	Mon: Course overview Wed: Python and Pandas	Data8 Chapters 3, 4, 5	
2	Jan 21, 23	Linear regression and classification	Covid trends (revisited) Classification examples	Mon: MLK day Wed: Regression concepts Fri: Classification	ISL Sections 3.1, 3.2, 3.5 Notes on regression ISL Sections 4.3, 4.4 Notes on classification	Assn 1 out
3	Jan 26,28	Stochastic gradient descent	SGD examples	Mon: Classification (continued) Wed: Stochastic gradient descent	ISL Section 6.2.2 ISL Section 10.7.2	Quiz 1
4	Feb 2, 4	Bias and variance, cross-validation	Bias-variance tradeoff Covid trends (revisited) California housing	Mon: Bias and variance Wed: Cross-validation	ISL Section 2.2 ISL Section 5.1	Assn 1 in Assn 2 out
5	Feb 9, 11	Tree-based methods and principal components	Trees and forests Visualizing trees	Mon: Trees Wed: Forests	ISL Sections 8.1, 8.2 ISL Section 12.2	Quiz 2
6	Feb 16, 18	PCA and dimension reduction	PCA demo 1 PCA demo 2 PCA dimension reduction Word embeddings	Mon: PCA Wed: Embeddings and language models	ISL Section 12.2	Assn 2 in Assn 3 out converter notebook
7	Feb 23, 25	Language models, Bayes	Bayesian inference	Mon: Language models Wed: Bayesian inference	OpenAI: Better language models Notes on Bayesian inference	Quiz 3

Where we're going

			embeddings			
7	Feb 23, 25	Language models, Bayes	Bayesian inference	Mon: Language models Wed: Bayesian inference	OpenAI: Better language models Notes on Bayesian inference	Quiz 3
8	March 2,4	Topic models Midterm exam (in class)	Topic models	Mon: Topic models Wed: Exam	On Canvas: Practice midterms / Sample solns Midterm / Sample soln	
9	Mar 23,25	Introduction to neural networks	Tensorflow playground Minimal neural network Regression examples	Mon: Neural networks Wed: Neural networks (continued)	ISL Sections 10.1, 10.2	Assn 3 in Assn 4 out
10	Mar 30, Apr 1	Reinforcement learning	Q-learning	Mon: Q-learning Wed: Reinforcement learning	Notes on backpropagation	Quiz 4
11	Apr 6, 8	Deep neural networks and LLMs	Autoencoder examples	Mon: Autoencoders Wed: LLMs	ISL Section 10.7	Assn 4 in Assn 5 out
12	Apr 13, 15	Transformers and LLMs	Attention GPT-4 Python API	Mon: Transformers Wed: LLM post-processing		Quiz 5
13	Apr 20, 22	Societal issues for machine learning		Mon: Panel discussion Wed: Course wrap up, iML Survivor		Assn 5 in
14	Monday May 4, 2pm	Final exam			Registrar: Final exam schedule Practice finals	

Outline: Neural networks

- Connecting to embeddings and logistic regression
- Alternative view of linear models
- Adding nonlinearities — activation functions
- Biological analogy and inspiration
- Backpropagation — high level
- Examples: Regression

Logistic Regression

Recall:

$$\log\left(\frac{P(y=1\,|\,x)}{P(y=0\,|\,x)}\right) = \beta^T x + \beta_0$$

Equivalently:

$$P(Y=1\,|\,x) \propto e^{\beta^T x + \beta_0}$$

Logistic Regression

In the multi-class case we have

$$P(Y = k \mid x) \propto e^{\beta_k^T x + \beta_{k0}}, \quad k = 1, \ldots, K - 1$$

We can write this in ML terminology as

$$\text{Softmax}\left(\left\{\beta_k^T x + \beta_{k0}\right\}\right)$$

Note: Can also use β_k for $k = \underline{0}, \ldots, K - 1$. This will be "overparameterized"

Logistic Regression

What if x is an image, represented as pixels? It might be hard to get an accurate classifier.

Want to learn *feature representation* $\phi(x)$.

The model becomes

$$P(Y = k \mid x) \propto e^{\beta_k^T \phi(x) + \beta_{k0}}, \quad k = 0, 1, \ldots, K-1$$

The parameters of ϕ and the parameters β need to be learned/trained.

Word embeddings

Applying this to language modeling, we could have a bigram model

The model becomes

$$P(v \mid w) \propto e^{\beta_v^T \phi(w)}$$

where $\phi(w)$ is a learned feature vector or "embedding vector" for each word.

The parameters β_v are also embeddings. By making them the same as ϕ we get the word2vec model.

Starting with regression

For linear regression, our loss function for an example (x, y) is

$$\begin{aligned}\mathcal{L} &= \frac{1}{2}(y - \beta^T x - \beta_0)^2 \\ &= \frac{1}{2}(y - f)^2\end{aligned}$$

where $f(x) = \beta^T x + \beta_0$.

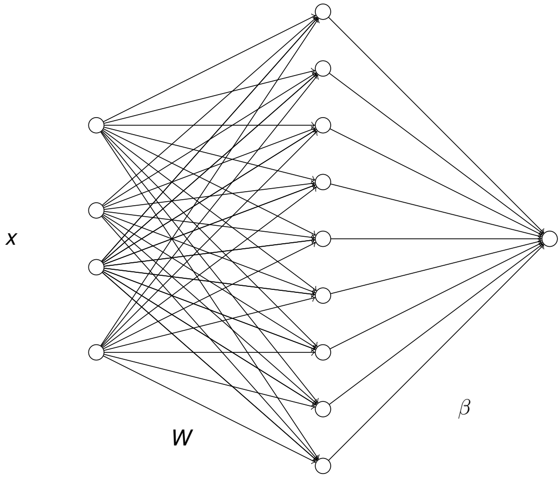
Adding a layer

Loss is

$$\mathcal{L} = \frac{1}{2}(y - f(x))^2$$

where now $f(x) = \beta^T h(x) + \beta_0$ where $h(x) = Wx + b$.

This can be viewed graphically.



x
W
β

Equivalent to linear model

But this is just a linear model

$$f = \widetilde{\beta}^T x + \widetilde{\beta}_0$$

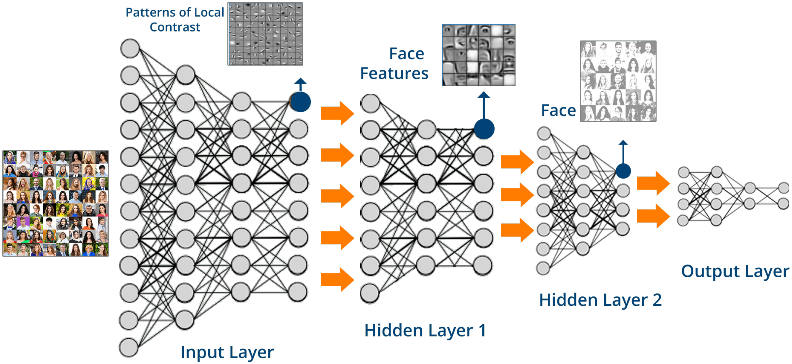
We get a reparameterization of a linear model; nothing new.

Need to add *nonlinearities*

Neural networks

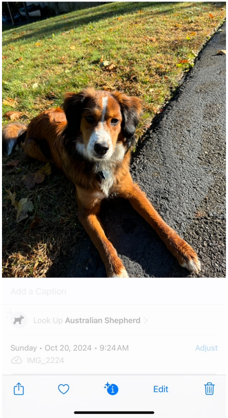
- Linear models are usually too simple to give good prediction rules
- Nonlinear models allow us to fit the data better—but run the risk of *overfitting*
- Neural networks are kind of in-between linear regression and k-nearest neighbors
- *Neural networks are layered linear models with carefully restricted nonlinearities*

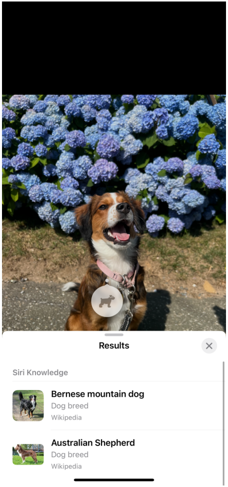
Deep neural networks

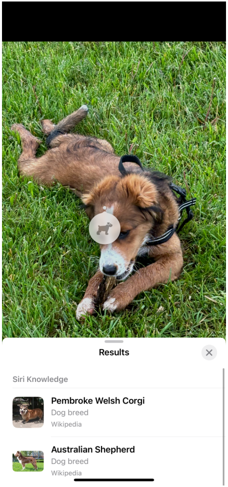


- Heuristics motivated from simplified view of the brain
- A particular form of nonlinear classification/regression

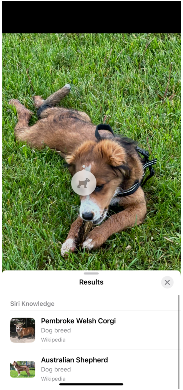
Everyday example







Everyday example

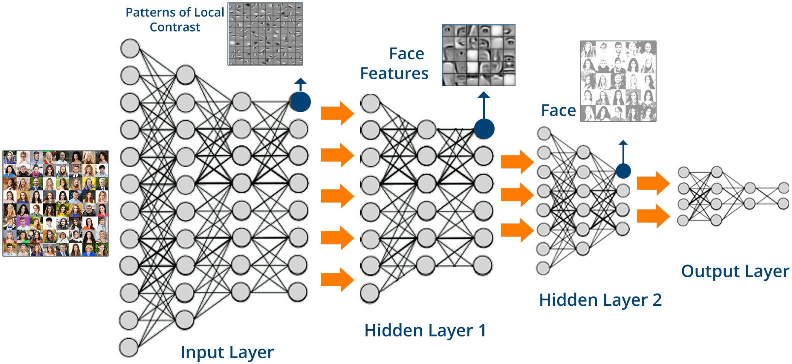


The neural network extracts “features” of the image that can be used to find similar images

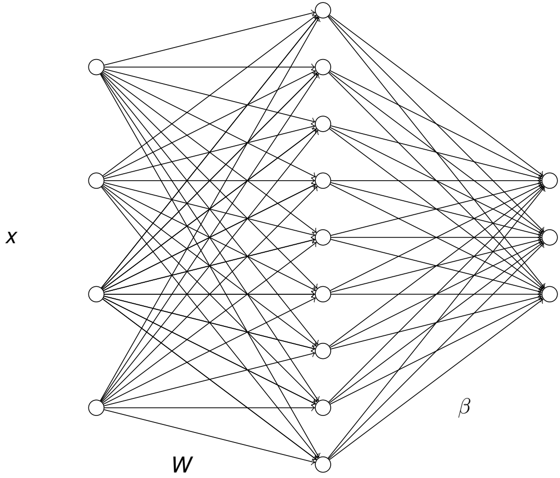
Everyday example

The neural network extracts “features” of the image that can be used to find similar images

Deep neural networks



- Heuristics motivated from simplified view of the brain
- A particular form of nonlinear classification/regression



This would be appropriate for Fisher iris data. Four inputs and classification with 3 classes—3 nodes in last layer.

Nonlinearities

Add *fixed nonlinearity*, for example

$$h = \max(Wx + b, 0)$$

applied component-wise to each "neuron"

- Typically the last layer is just linear
- So, a neural net is a linear model, but with complex *predictor variables* or "features" that are automatically learned from data

Nonlinearities

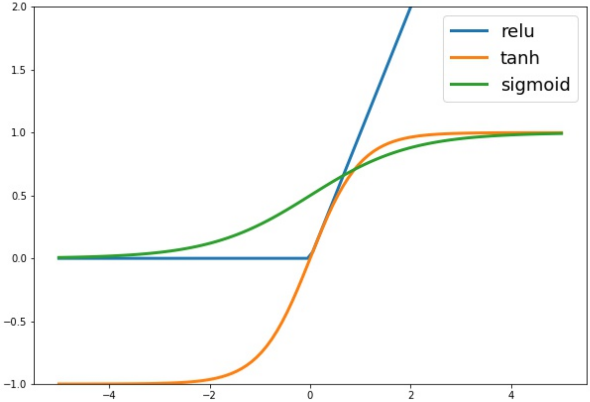
Commonly used nonlinearities:

$$\phi(u) = \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

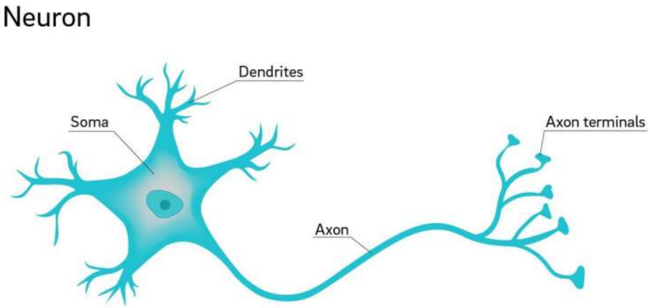
$$\phi(u) = \text{sigmoid}(u) = \frac{1}{1 + e^{-u}}$$

$$\phi(u) = \text{relu}(u) = \max(u, 0)$$

Nonlinearities

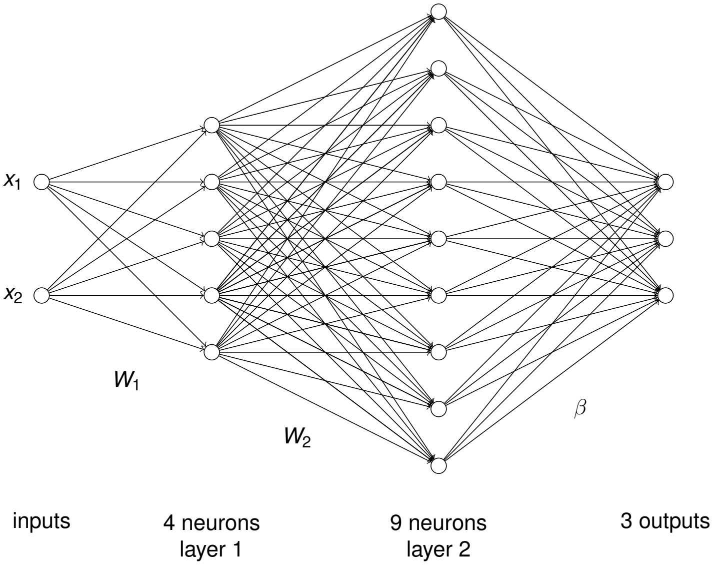


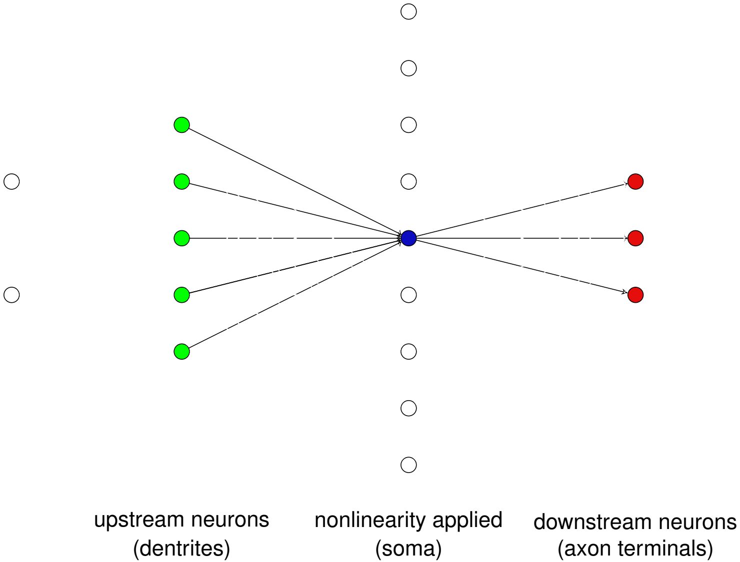
Biological analogy



Biological analogy

- The dendrites play the role of inputs, collecting signals from other neurons and transmitting them to the soma, which is the “central processing unit.”
- If the total input arriving at the soma reaches a threshold, an output is generated — this is the nonlinearity!
- The axon is the output device, which transmits the output signal to the dendrites of other neurons.

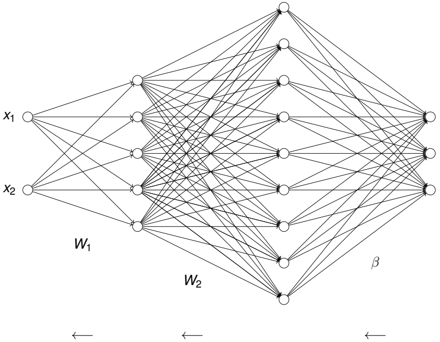

x_1
x_2
W_1
W_2
β
inputs
4 neurons
layer 1
9 neurons
layer 2
3 outputs


upstream neurons
(dentrites)
nonlinearity applied
(soma)
downstream neurons
(axon terminals)

Training

- The parameters are trained by “stochastic gradient descent”
  - Stream through the data, using small batches
  - Add a little bit to each network weight — proportional to how sensitive the predictions are to that weight (derivative)
  - Derivatives calculated with automatically — no math!
  - Stop if overfitting is detected!

High level idea: Backpropagation



Start at last layer, send error information back to previous layers

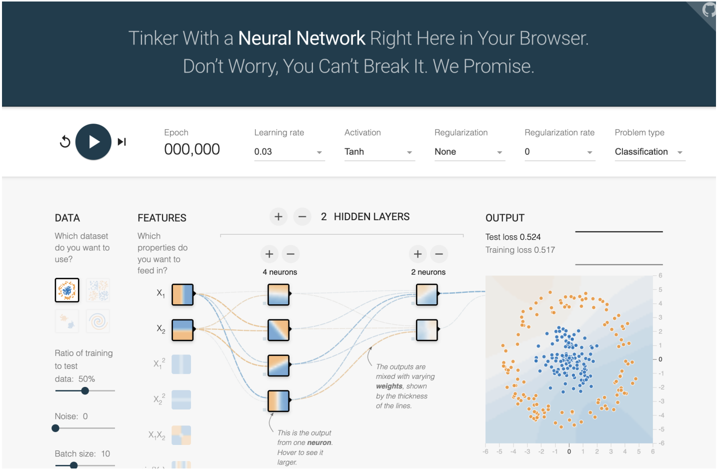
---

First described by Geoff Hinton, who won the Nobel Prize in Physics a couple weeks ago.

These types of neural networks are called "multilayer perceptrons" (MLPs)

They can be surprisingly robust to overfitting!

Demo



https://playground.tensorflow.org/

Summary

- For complex data we may want to learn features of the inputs
- This representation can be part of a logistic regression model
- Features that are linear transforms followed by a nonlinearity form the building blocks of (artificial) neural networks
- Based on a crude analogy with neurons in biological brains
- Trained using stochastic gradient descent
- Can be thought of as a particular type of nonlinear regression model